

Integration Report

Gianluca Bresolin
gianluca.bresolin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Tommaso Prandin
tommaso.prandin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Abstract

This *Integration Report* includes some additional details regarding some aspects of the project that were not covered in the *Technical Report*.

CCS Concepts

• **Software and its engineering** → **Real-time systems software**; **Embedded software**; *Software architectures*.

Keywords

Real-Time Kernels, Real-Time Systems, Embedded Software, Software Architectures

ACM Reference Format:

Gianluca Bresolin and Tommaso Prandin. 2025. Integration Report. In *Proceedings of Real-Time Kernels and Systems Exam (RTKS '25)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Why we used a one-to-one mapping between tasks and watchdogs in the Rust System?

Since *RTIC framework* does not provide any built-in mechanism to handle task deadlines, we decided to use a one-to-one mapping between tasks and watchdogs. This way, each watchdog is responsible for monitoring the deadline of a single task, making the implementation simpler and more straightforward.

Besides the simplicity of implementation, this approach was chosen to avoid the runtime cost of managing a pool of deadlines and `deadline_protected_objects` at each deadline check. In fact, an alternative implementation of a single watchdog would require identifying the earliest next deadline among all the monitored ones at each deadline check, in order to delay its execution accordingly. This operation is also needed at every task activation, since it could introduce a new earliest deadline.

Even if we could optimize this operation by using an appropriate data structure, such as a min-heap, the one-to-one mapping approach, instead, allows to manage each watchdog independently, without the need of any search, insert or delete operation over a pool of objects. This results in a constant-time complexity for each deadline check and task activation, thus reducing the runtime overhead, improving the system's responsiveness and reducing

watchdogs' interference over system tasks.

Moreover, since the *RTIC framework* requires declaring for each task locals and shared resources at compile time, by using a one-to-one mapping we can leverage the single responsibility principle, enhancing code readability and maintainability against a more complex single watchdog implementation.

However, this approach introduces a higher activation cost, since each watchdog consists in a task, and as such it has its own activation overhead. This is a price that we considered more acceptable than the previous described runtime cost, since the activation cost is paid only once and does not affect the system responsiveness. Activation cost is not the only negative aspect of this approach: by having a one-to-one mapping, we also introduce a memory overhead, since each watchdog task requires its own dedicated space for the asynchronous task executor. This could be a problem in systems with many tasks, where the memory overhead could become significant. In our case, given the limited number of tasks in the system, we considered this overhead acceptable, but we acknowledge that in systems with a larger number of tasks, this could become a trade-off to consider carefully against the runtime cost of a single watchdog implementation.

2 How we assign priorities in the system?

The priority assignment in our system is intended to enable high system utilization while ensuring feasibility (i.e., all tasks meet their deadlines on time), under *Fixed-Priority Preemptive Scheduling (FPFS)*.

Task	Type	Period/Min-InterArr (ms)	Relative Deadline (ms)
Activation Log Reader	Sporadic	3000	1000
On Call Producer	Sporadic	5000	800
Regular Producer	Periodic	1000	500
External Event Server	Sporadic (interrupt)	5000	100

Table 1: Task Table

The system consists of four tasks: `regular_producer`, `on_call_producer`, `activation_log_reader` and `external_event_server` (watchdogs are not included in this priority analysis, since they are always assigned the highest priority for reasons already discussed in the *Technical Report*, section 5 [1]). While `external_event_server` is independent of the other tasks, the other three are characterized by precedence relationships. Specifically, `regular_producer` is responsible for activating both `activation_log_reader` and `on_call_producer`, based on different activation criteria.

According to these relationships and on the task time parameters reported in Table [1], we can derive the following constraints:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
RTKS '25, Padova, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

- `external_event_server` must have the highest priority upon all the other forementioned tasks. This is because it has the shortest relative deadline (100 ms) and it is a sporadic task. Therefore, it must be able to respond quickly when activated, preempting other tasks if necessary. Since it is independent of the other tasks, the underlying principle is the same as for *Deadline Monotonic* scheme;
- `regular_producer` must have a higher priority than both `activation_log_reader` and `on_call_producer`. This is because it activates both tasks and has a shorter relative deadline (500 ms) than both (1000 ms and 800 ms, respectively). Therefore, it must be able to complete its execution before the other two tasks are dispatched, so that the activated tasks can be scheduled according to their own deadlines rather than being constrained by the `regular_producer`'s deadline;
- `on_call_producer` must have a higher priority than `activation_log_reader`. This is because it has a shorter relative deadline (800 ms) than `activation_log_reader` (1000 ms). Therefore, it must be able to complete its execution before `activation_log_reader` is dispatched. It's important to note that this constraint is valid only because there is a `regular_producer` job activating both tasks (as described in the *Technical Report*, section 7.2 [1]). Otherwise, it would just be sufficient to have a priority lower than `regular_producer`'s one, since there would not be any interference between `on_call_producer` and `activation_log_reader`.

The principle underlying these constraints is to ensure that tasks with shorter relative deadlines are given higher priorities (as in *Deadline Monotonic* scheme), while also taking into account the relationships between tasks.

Based on these constraints, we assigned the following priorities:

- `external_event_server`: Priority 11 (highest);
- `regular_producer`: Priority 7;
- `on_call_producer`: Priority 5;
- `activation_log_reader`: Priority 3 (lowest).

References

- [1] Prandin T. Bresolin G. 2025. Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime. (2025).